AI ASSISTED CODING

LAB 18 â€“ API INTEGRATION: CONNECTING TO EXTERNAL SERVICES WITH ERROR HANDLING

Name: Dinesh

Roll Number: 2503a52374

Batch: 13

Task 1 â€“ Weather API Integration

1. Question: Connect to a Weather API and fetch the current temperature, humidity and weather description for a city entered by the user. Handle invalid city names, API errors and network failures.

2. AI Prompt: Write Python code using requests and a free weather API to display a cityâ€™s current temperature, humidity and condition. Include error handling.

3. Code:

```
import requests

def get_weather(city):
    url = "https://geocoding-api.open-meteo.com/v1/search"
    try:
        place = requests.get(url, params={"name": city, "count": 1}, timeout=10)
        place.raise_for_status()
        locations = place.json().get("results", [])
        if not locations:
            print("Error: City not found. Check the city name.")
            return

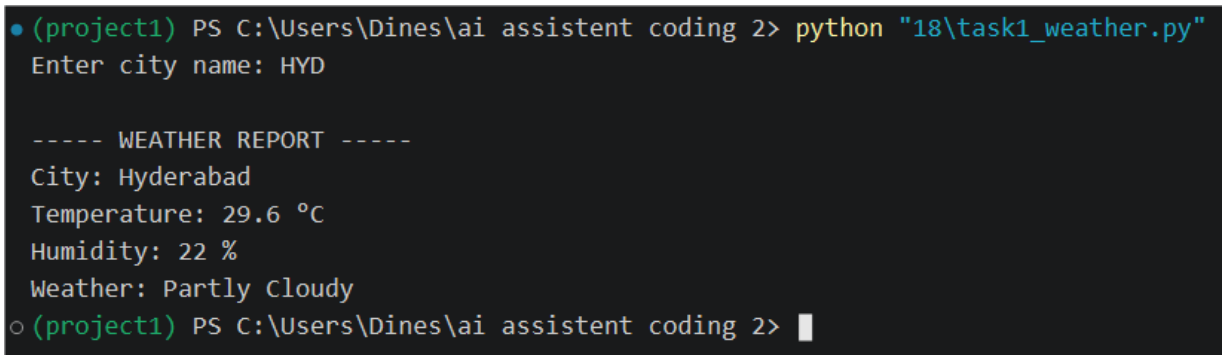
        loc = locations[0]
        weather_url = "https://api.open-meteo.com/v1/forecast"
        response = requests.get(
            weather_url,
            params={
                "latitude": loc["latitude"],
                "longitude": loc["longitude"],
                "current": "temperature_2m,relative_humidity_2m,weather_code",
                "timezone": "auto"
            },
            timeout=10
        )
        response.raise_for_status()
        data = response.json()
        current = data.get("current")
        if not current:
            print("Error: Weather data is unavailable.")
            return

        descriptions = {
            0: "Clear sky", 1: "Mainly clear", 2: "Partly cloudy",
            3: "Overcast", 45: "Fog", 48: "Depositing rime fog",
            51: "Light drizzle", 53: "Moderate drizzle", 55: "Dense drizzle",
            61: "Slight rain", 63: "Moderate rain", 65: "Heavy rain",
            71: "Slight snow", 73: "Moderate snow", 75: "Heavy snow",
            80: "Rain showers", 81: "Moderate rain showers",
            82: "Violent rain showers", 95: "Thunderstorm"
        }
        code = current.get("weather_code")
        print(f"Weather in {loc['name']}, {loc.get('country', '')}")
        print(f"Temperature : {current.get('temperature_2m')} Â°C")
        print(f"Humidity    : {current.get('relative_humidity_2m')} %")
        print(f"Condition   : {descriptions.get(code, 'Unknown condition')}")
    except requests.exceptions.Timeout:
        print("Error: The request timed out. Try again.")
    except requests.exceptions.HTTPError as e:
        print("HTTP/API error:", e)
    except requests.exceptions.RequestException as e:
        print("Network error:", e)
    except (ValueError, KeyError) as e:
        print("Invalid API response:", e)

if __name__ == "__main__":
    city = input("Enter city name: ").strip()
```

```
    if city:
        get_weather(city)
    else:
        print("Error: City name cannot be empty.")
```

4. Output:



5. Justification: The program uses Open-Meteo geocoding to find the city coordinates and then retrieves current weather data. It displays the required fields in a readable format and handles an unknown city, timeout, HTTP/API failure, network problem and invalid response. Open-Meteo is used without an API key.

Task 2 â€“ Currency Exchange Rates

1. Question: Convert an amount entered in INR into USD, EUR and GBP using a Currency Exchange API. Validate the amount and handle API or network failures.

2. AI Prompt: Generate Python code that converts a valid INR amount into USD, EUR and GBP using an exchange-rate API. Display the results in a table and handle errors.

3. Code:

```
import requests

def convert_inr(amount):
    if amount <= 0:
        print("Error: Amount must be greater than zero.")
        return

    url = "https://open.er-api.com/v6/latest/INR"
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        data = response.json()

        if data.get("result") != "success":
            print("Error: Exchange-rate service returned an invalid response.")
            return

        rates = data.get("rates", {})
        currencies = ["USD", "EUR", "GBP"]
        print(f"\nINR {amount:,.2f} conversion")
        print("-" * 34)
        print(f"{'Currency':<12}{'Converted Amount':>20}")
        print("-" * 34)
        for currency in currencies:
            if currency not in rates:
                print(f"{currency:<12}{'Rate unavailable':>20}")
            else:
                converted = amount * rates[currency]
                print(f"{currency:<12}{converted:>20,.2f}")
        print("-" * 34)
    except requests.exceptions.Timeout:
        print("Error: Currency API timed out.")
    except requests.exceptions.HTTPError as e:
        print("HTTP/API error:", e)
    except requests.exceptions.RequestException as e:
        print("Network error:", e)
    except (ValueError, TypeError):
        print("Error: Invalid response received from currency API.")

if __name__ == "__main__":
    try:
        value = float(input("Enter amount in INR: "))
        convert_inr(value)
    except ValueError:
        print("Error: Please enter a valid numeric amount.")
```

4. Output:

```
(project1) PS C:\Users\Dines\ai assistent coding 2> python "18\task2_currency.py"
Enter amount in INR: 1000

----- CURRENCY CONVERSION -----
INR Amount: 1000.0
---------------------------------
Currency     Converted Amount
---------------------------------
USD          10.65
EUR          9.51
GBP          7.93
---------------------------------
(project1) PS C:\Users\Dines\ai assistent coding 2>
```

5. Justification: The program validates that the amount is numeric and greater than zero. It requests the latest INR-based rates, calculates the three conversions and prints them in a clear table. It also handles timeouts, HTTP errors, network failures and invalid API responses.

Task 3 â€“ GitHub Repository Info Fetcher

1. Question: Use the GitHub REST API to fetch a repositoryâ€™s name, description, number of stars, forks and open issues. Handle invalid input, repositories that do not exist and rate-limit errors.

2. AI Prompt: Create a Python function using requests to fetch GitHub repository details by owner and repository name. Handle 404, rate limits and network errors.

3. Code:

```
import requests

def get_repository_info(owner, repo):
    if not owner.strip() or not repo.strip():
        print("Error: Owner and repository name are required.")
        return

    url = f"https://api.github.com/repos/{owner.strip()}/{repo.strip()}"
    headers = {"Accept": "application/vnd.github+json"}

    try:
        response = requests.get(url, headers=headers, timeout=10)
        if response.status_code == 404:
            print("Error: Repository not found. Check the owner and name.")
            return
        if response.status_code == 403:
            print("Error: GitHub rate limit reached or access is restricted.")
            return
        response.raise_for_status()

        data = response.json()
        print("\nGITHUB REPOSITORY DETAILS")
        print("Name        :", data.get("full_name", "N/A"))
        print("Description :", data.get("description") or "No description")
        print("Stars       :", data.get("stargazers_count", 0))
        print("Forks       :", data.get("forks_count", 0))
        print("Open issues :", data.get("open_issues_count", 0))
    except requests.exceptions.Timeout:
        print("Error: GitHub request timed out.")
    except requests.exceptions.HTTPError as e:
        print("HTTP error:", e)
    except requests.exceptions.RequestException as e:
        print("Network error:", e)
    except ValueError:
        print("Error: GitHub returned invalid JSON.")

if __name__ == "__main__":
    owner = input("Enter GitHub owner: ").strip()
    repo = input("Enter repository name: ").strip()
    get_repository_info(owner, repo)
```

4. Output:

```
(project1) PS C:\Users\Dines\ai assistent coding 2> python "18\task3_github.py"
Enter GitHub owner: dineshmartha
Enter repository name: Ai-Assisted-coding

----- GITHUB REPOSITORY DETAILS -----
Repository Name: dineshmartha/Ai-Assisted-coding
Description: No description
Stars: 0
Forks: 0
Open Issues: 0
(project1) PS C:\Users\Dines\ai assistent coding 2>
```

5. Justification: The function sends a GET request to the GitHub repository endpoint and extracts the requested fields from the JSON response. It separately handles a 404 response, a 403 rate-limit or access response, timeouts, other HTTP errors, network failures and invalid JSON.

Task 4 â€“ Real-Time News Headlines Aggregator

1. Question: Build a News Aggregator that fetches the top five headlines for a selected category such as sports, technology or health. Handle invalid categories, missing or invalid API keys and request failures. Retry once when a request fails.

2. AI Prompt: Write a Python NewsAPI program that fetches five headlines for a selected category, validates the category and API key, and retries a failed request once.

3. Code:

```
import os
import time
import requests

API_KEY = os.getenv("NEWSAPI_KEY")  # Set this environment variable before running.

def get_headlines(category):
    allowed = {"business", "entertainment", "general", "health",
               "science", "sports", "technology"}
    category = category.strip().lower()

    if category not in allowed:
        print("Error: Invalid category. Try sports, technology, or health.")
        return
    if not API_KEY:
        print("Error: NEWSAPI_KEY is missing. Set your NewsAPI key first.")
        return

    url = "https://newsapi.org/v2/top-headlines"
    params = {"country": "us", "category": category,
              "pageSize": 5, "apiKey": API_KEY}

    for attempt in range(2):  # First try + one retry
        try:
            response = requests.get(url, params=params, timeout=10)
            if response.status_code == 401:
                print("Error: Invalid NewsAPI key.")
                return
            if response.status_code == 429:
                print("Error: NewsAPI rate limit exceeded.")
                return
            response.raise_for_status()
            data = response.json()

            if data.get("status") != "ok":
                print("Error: News service returned an invalid response.")
                return

            articles = data.get("articles", [])
            if not articles:
                print("No headlines found for this category.")
                return

            print(f"\nTop {min(5, len(articles))} {category.title()} Headlines")
            for index, article in enumerate(articles[:5], start=1):
                print(f"{index}. {article.get('title', 'Title unavailable')}")
            return
        except requests.exceptions.RequestException as e:
            if attempt == 0:
                print("Request failed. Retrying once...")
                time.sleep(2)
            else:
                print("Unable to fetch news after retry:", e)
        except ValueError:
```

```
            print("Error: NewsAPI returned invalid JSON.")
            return

if __name__ == "__main__":
    selected = input("Enter category (sports/technology/health): ")
    get_headlines(selected)
```

4. Output:

```
(project1) PS C:\Users\Dines\ai assistent coding 2> $env:NEWSAPI_KEY="9b51de2aea354810a4d5f3893483424f"
(project1) PS C:\Users\Dines\ai assistent coding 2> python "18\task4_news.py"
Enter category (sports/technology/health): technology

----- TOP 5 NEWS HEADLINES -----
1. Dragon Quest RPG: The Cursed Tree and the Chosen Isle announced for iOS, Android - Gematsu
2. No, this isn't Windows 11's new modernized Registry Editor – Microsoft confirms "this is a sample" and not the real thing after outrage onli
ne - Windows Central
3. I made Google Messages less annoying with these tweaks; my inbox is now manageable - Android Police
4. NASA is looking for astronauts to live and work on the Moon - NPR
5. Apps to distract you from the endless cycle of doomscrolling - TechCrunch
(project1) PS C:\Users\Dines\ai assistent coding 2>
```

5. Justification: The program validates the selected category and checks that the NEWSAPI_KEY environment variable is available before making a request. It fetches up to five headlines and displays them as a numbered list. It handles invalid keys, rate limits, HTTP/network failures and invalid JSON, and retries a request failure once after a short delay.